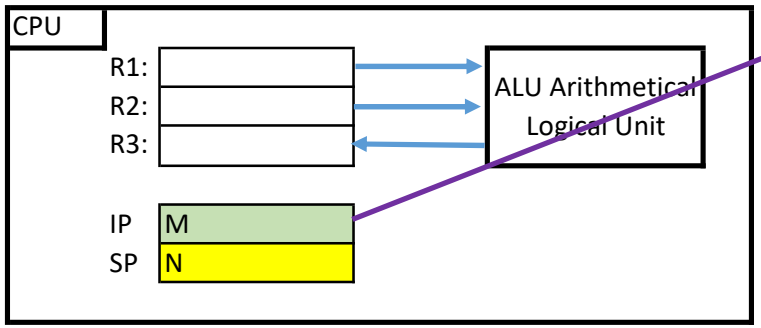
Increment variable v: v++;

1 IP points to current instruction: INCrement variable v

Adresse	0	1	…	Variable v	…	M	M+1	M+2			N-2	N-1	N
Memory:				123		INC	v	…	…				



Description:

R1, R2, R3: General purpose register

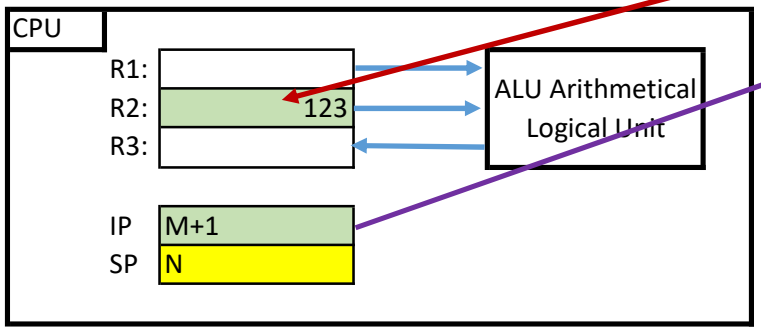
ALU: Does the maths and comparisons

IP: Inst programm ("Current instruction")

SP: Stack Pointer, not used here

2 Execution: Content of v is transfered to register R2

Adresse	0	1	…	Variable v	…	M	M+1	M+2			N-2	N-1	N
Memory:				123		INC	v	…	…				

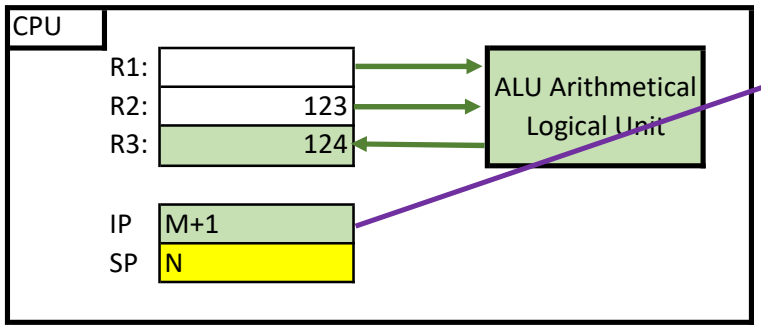


The value of variable v is moved to register R2

Any value in R2 is overwritten

3 Execution: increment register R2

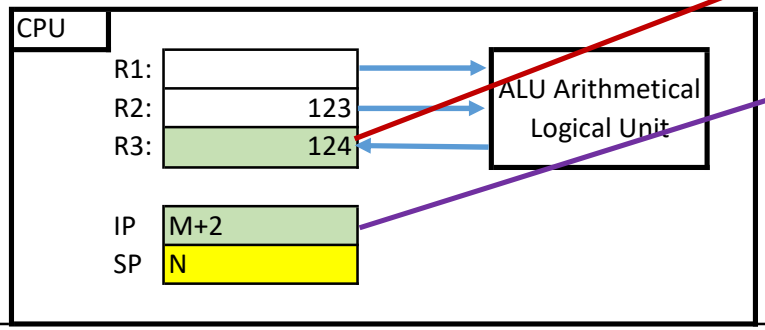
Adresse	0	1	…	Variable v	…	M	M+1	M+2			N-2	N-1	N
Memory:				123		INC	v	…	…				



ALU is triggered to do the maths, increment R2
Result is stored in R3
Any value in R3 is overwritten

4 Execution: Store R3 back to variable v and carry on (IP++)

Adresse	0	1	…	Variable v	…	M	M+1	M+2	M+3		N-2	N-1	N
Memory:				124		INC	v	…	…				



After finishing the instruction INCrement,
incl. moving result back to v,
the contents of R2 and R3 are "garbage"

The instruction v++; was introduced because early compilers could not map v = v + 1; to the INC assembler statement.
Instead, they mapped it to the ADD statement :(. The ADD assembler statement needs more time.
Nowadays, compilers are so smart and map v+= 1; to INC. The v++ is therefore obsolete and makes code unreadable:
v[i++] += 2; vs v[++i] += 2; whats the difference at a glance? Quality and maintenance people are not amused!
😂 **Unfortunately, teachers regard ++ as the pinnacle of digitalization … and are very proud of this.**